

The GLP–Mono Architecture

A Monolithic Paged Grammar–Lemma Language Model

Joachim De Witte

July 2026

©2026 Joachim De Witte. All rights reserved.

Published under the Creative Commons BY–NC–ND 4.0 license.

Abstract

GLP–Mono is a monolithic next-token language model built around deterministic grammar tokens, paged lemma clusters, and fixed-dimensional output geometry. Unlike transformer-based architectures, which rely on self-attention over high-dimensional hidden states and a single global vocabulary, GLP–Mono performs generation through a single feedforward network whose output is split into a learned lemma embedding and a one-hot page selector. Each page contains a compact set of grammar–lemma token pairs discovered during training, while terminal grammar tokens are implicitly available in every page.

Next-token prediction proceeds by selecting the highest-scoring page (with fallback), ranking all token pairs in that page by cosine similarity to the predicted lemma embedding, and sampling from the top- k candidates. Context is represented through grammar-driven compression, sentence-level embeddings, and a compact recurrent narrative memory, yielding a fixed-dimensional input vector without attention. Grammar tokens remain symbolic and deterministic, providing structural cadence, while lemma tokens are represented as learned output embeddings updated through a damped mixing rule.

GLP–Mono therefore constitutes a structurally governed, domain-isolated alternative to transformer LMs: a single MLP with deterministic grammar, learned lemma geometry, context-driven paging, and stable next-token behaviour without reliance on large embedding spaces or attention mechanisms.

1 Significance of the GLP–Mono Architecture

GLP–Mono occupies a distinct position within the broader landscape of language model architectures. It does not aim to compete with large transformer-based models in scale, fluency, or dataset coverage. Instead, GLP–Mono demonstrates that coherent next-token behaviour, domain-faithful world geometry, and interpretable generative structure can emerge from a compact architecture built around deterministic grammar, paged lemma organization, and fixed-dimensional context encoding. Its significance lies not in raw generative power, but in the architectural principles it makes possible.

1.1 A Structured Alternative to Attention-Based Models

Contemporary language models rely almost universally on self-attention over high-dimensional hidden states. GLP–Mono shows that this paradigm is not the only viable route to coherent generation. By combining symbolic grammar tokens, paged lexical clusters, cosine-based ranking, and a monolithic MLP, GLP–Mono provides a structurally governed alternative in which:

- grammar remains deterministic and interpretable,

- lemma geometry is compact and domain-isolated,
- paging replaces global softmax over large vocabularies,
- context is encoded without attention,
- long-horizon coherence arises from recurrent narrative memory.

This architecture demonstrates that explicit structure can coexist with learned lexical geometry, yielding a model that is both inspectable and expressive at small scale.

1.2 Successor to DMLG and Earlier GLP Variants

GLP-Mono inherits the structural insights of DMLG—deterministic grammar, paging, recursive evaluation, and IEGR completeness—while resolving the limitations of modular micro-model ensembles. Earlier GLP variants employed a dual grammar/lemma network that restricted generative performance. GLP-Mono collapses this duality into a single monolithic network, enabling stronger next-token behaviour while preserving the structural cadence of its predecessors.

Although GLP-Mono remains small-scale, trained on limited datasets with modest compute budgets, it provides a more LM-like generative core than DMLG. It is therefore the natural architectural successor: structurally grounded, compact, and capable of producing domain-faithful proto-stories suitable for downstream refinement.

1.3 Hybrid Generative Systems and Domain-Bound AGI

The significance of GLP-Mono becomes most apparent in hybrid systems. When paired with a transformer LLM, GLP-Mono acts as a deterministic canon engine: it supplies world geometry, narrative motifs, and structural attractors, while the LLM supplies stylistic refinement and narrative shaping. Because GLP-Mono encodes the full canonical ontology of its training domain, the LLM can reproduce this canon *without any world prompting*. The GLP-generated proto-story implicitly carries the world model, and the LLM merely rewrites it into fluent natural language.

This hybrid configuration enables a form of domain-bound AGI: a system in which world reasoning, narrative coherence, and stylistic expression emerge from the interaction between a structured LM and a large transformer. The narrator pipeline serves as a fully worked example of this paradigm, illustrating how GLP-Mono can steer a general-purpose LLM through context rather than parameter modification.

1.4 Scope and Ecosystem Boundaries

Scaling GLP-Mono to the size of contemporary LLMs—billions of parameters, massive datasets, and industrial training budgets—is outside the scope of the ecosystem. The ecosystem is built around unification, structural clarity, and compact world engines rather than large-scale pretraining. GLP-Mono is designed to be small, deterministic, and domain-stable, providing a canonical substrate for hybrid generative systems rather than serving as a standalone large LM.

1.5 Summary

GLP-Mono is significant not because it rivals transformers in scale, but because it demonstrates a different architectural path: one in which deterministic grammar, paged lexical geometry, and compact context encoding yield coherent next-token behaviour and stable world models. Its role as a canon engine in hybrid systems highlights the value of structured LM architectures and provides a foundation for domain-bound generative intelligence within the broader ecosystem.

2 Introduction

Contemporary language models are dominated by transformer-based architectures that unify grammar, lexical content, and contextual structure into a single high-dimensional embedding space governed by self-attention. While these systems achieve strong generative fluency, their internal organization remains monolithic: all tokens share the same embedding geometry, vocabulary size determines output dimension, and long-horizon coherence depends on large hidden states and deep attention stacks. As a result, transformer LMs provide no explicit structural backbone, no deterministic grammar regime, and no mechanism for domain-isolated lexical organization.

GLP-Mono introduces a fundamentally different approach to next-token language modeling. Instead of attention over a global vocabulary, GLP-Mono employs a single feedforward network whose output is split into two components: a fixed-dimensional lemma embedding and a one-hot page selector. Each page contains a compact set of grammar-lemma token pairs discovered during training, while terminal grammar tokens are implicitly available in every page. Next-token prediction is performed by selecting the highest-scoring page (with fallback), ranking all token pairs in that page by cosine similarity to the predicted lemma embedding, and sampling from the top- k candidates.

Context is represented through grammar-driven compression, sentence-level embeddings, and a compact recurrent narrative memory. This yields a fixed-size input vector without attention, enabling stable long-horizon behaviour even at small scale. Grammar tokens remain symbolic and deterministic, providing structural cadence, while lemma tokens are represented as learned output embeddings updated through a damped mixing rule. Paging acts as a sparse routing mechanism that isolates domain-specific token clusters and prevents cross-domain drift.

The result is a structurally governed, domain-isolated language model whose geometry is fixed, whose grammar is deterministic, and whose next-token behaviour emerges from a combination of learned lemma embeddings and context-driven page activation. GLP-Mono demonstrates that coherent language generation does not require large embedding spaces or attention mechanisms: a single monolithic MLP, paired with deterministic grammar and paged lexical organization, is sufficient to produce stable, expressive, and interpretable next-token dynamics.

3 From DMLG to GLP-Mono

GLP-Mono is the architectural successor to the Dynamic Modular Language Graph (DMLG). Both systems share a structural lineage: deterministic grammar tokens, paged lexical organization, context-driven state, and recursive mechanisms for sentence-level validation. However, GLP-Mono represents a decisive shift toward a monolithic next-token language model that aligns more closely with mainstream LM design while preserving the structural governance principles of DMLG.

3.1 DMLG as a Structural Precursor

DMLG introduced a graph-based generative substrate in which grammar, lemma structure, paging, and evaluator-governed recursion formed a deterministic backbone for controlled text generation. Its architecture was IEGR-complete: it possessed explicit geometry, recursive evaluation loops, and domain-isolated paging. Yet DMLG operated at small scale: limited datasets, low energy budgets, micro-model ensembles, and short training cycles. As a result, its generative output was structurally coherent but not competitive with large transformer models in fluency or stylistic richness.

DMLG demonstrated that structured generation was possible without attention, but its modular micro-model design constrained its expressive capacity. The dual-network grammar/lemma architecture of early GLP variants inherited these limitations: strong structural cadence, but reduced generative performance.

3.2 GLP-Mono as a Generative Successor

GLP-Mono resolves these constraints by collapsing the architecture into a single monolithic MLP with deterministic grammar and learned lemma geometry. Paging remains central, but routing is now driven by a unified network rather than distributed micro-models. Context encoding retains the structural ideas of DMLG—history compression, recurrent narrative memory, and grammar-driven sentence embeddings—yet GLP-Mono learns how these components interact rather than relying on evaluator-based traversal.

This shift yields a model that is architecturally compact, structurally interpretable, and generatively stronger than its predecessors. GLP-Mono does not match the fluency of large transformer LMs—its dataset, training budget, and model size remain intentionally small—but it provides a coherent, domain-faithful generative substrate that behaves like a true language model rather than a modular graph engine.

3.3 Hybrid Strength in the Narrator Pipeline

The combination of GLP-Mono and a transformer LLM enables a form of hybrid generation that neither architecture can achieve alone. GLP-Mono supplies the world geometry, canonical grammar, and narrative attractors; the LLM supplies stylistic refinement, narrative shaping, and quality control. Because GLP-Mono produces structurally coherent proto-stories, the LLM can reproduce the full canon of the GLP-trained domain *without any world prompting*. The world is implicitly encoded in the GLP output, and the LLM merely rewrites it into fluent natural language.

This hybrid configuration demonstrates the architectural value of GLP-Mono: a compact, deterministic canon engine that can steer a large LLM through context rather than parameter modification. It preserves the structural insights of DMLG while providing a more expressive and LM-like generative core, enabling structured long-form generation in downstream systems such as the narrator pipeline.

4 Architecture

GLP-Mono is a monolithic next-token language model built around deterministic grammar tokens, paged grammar-lemma token pairs, and fixed-dimensional output geometry. The architecture consists of four tightly integrated components: a context encoder, a recurrent narrative memory, a single feedforward network with learned activation, and a paged output space containing all grammar-lemma pairs observed during training. Together, these components form a deterministic, domain-isolated next-token generator without attention, alignment, or auxiliary control mechanisms.

4.1 Context Encoding

At each generative step, GLP-Mono receives a fixed-dimensional context vector constructed from three sources:

1. **History embeddings:** A sliding window of previously generated sentences is encoded into large and medium embeddings using a grammar-driven sentence encoder. These embeddings compress structural information such as part-of-speech cadence and lemma distribution.
2. **Current-sentence embedding:** Lemma tokens in the partially generated sentence are represented through clipped medium embeddings, combined with a normalized position scalar.
3. **Narrative memory:** A compact recurrent state tracks actors, actions, moments, and atmosphere using GRU-like updates over learned lemma embeddings. This state provides long-horizon coherence without attention.

The concatenation of these components yields a single fixed-size input vector supplied to the monolithic network.

4.2 Monolithic Network with Learned Activation

GLP-Mono employs a single feedforward network:

$$f(x) = W_3 \sigma(W_2 \sigma(W_1 x)),$$

where σ is a learned activation function implemented as a small three-layer MLP. This activation MLP enables expressive non-linear behaviour while preserving deterministic geometry. The network outputs a vector split into two parts:

1. a **lemma embedding** in $\mathbb{R}^d$,
2. a **page-logit vector** in $\mathbb{R}^P$.

The lemma embedding is normalized to unit length. The page logits determine which page of grammar-lemma token pairs is active for the next-token decision.

4.3 Paged Output Space

During training, GLP-Mono constructs a set of pages. Each page contains:

- a set of input lemma keys used for routing,
- a set of grammar-lemma token pairs (G, L) observed during training.

Terminal tokens (`<PERIOD>`, `<COMMA>`, `<EXCLAMATION>`, `<QUESTION>`, `<EOL>`) are implicitly present in every page and require no routing. Pages act as sparse MoE-style clusters: only the highest scoring page (with fallback) is used for next-token selection.

4.4 Next-Token Selection

Next-token prediction proceeds as follows:

1. The monolithic network outputs a lemma embedding $\hat{e}$ and page logits.
2. The highest-scoring page is selected; if it contains no valid token pairs, the model falls back to the next page.
3. All grammar-lemma pairs (G, L) in the active page, plus the five terminal tokens, are scored by cosine similarity:

$$s(G, L) = \cos(\hat{e}, e_{G,L}),$$

where $e_{G,L}$ is the learned output embedding for the pair.

4. The ranked list is truncated to the top- k candidates and sampled using temperature-scaled softmax.

Grammar tokens are deterministic symbols; lemma tokens are learned embeddings. No grammar-lemma filtering is applied: compatibility is encoded implicitly in the page structure learned during training.

4.5 Learned Output Embeddings

Each grammar-lemma pair (G, L) has a learned output embedding updated through a damped mixing rule after each training batch:

$$e_{G,L} \leftarrow (1 - \alpha)e_{G,L} + \alpha\hat{e},$$

where α decreases gradually over training. This mechanism allows rapid early learning while ensuring long-term stability and preventing drift in the paged output space.

4.6 Deterministic Geometry

GLP-Mono maintains fixed geometry throughout training and inference:

- grammar tokens are symbolic and deterministic,
- lemma embeddings have fixed dimension,
- pages provide domain-isolated clusters,
- the monolithic network has fixed parameter count,
- vocabulary growth does not alter model geometry.

This deterministic structure enables stable multi-world blending, predictable long-horizon behaviour, and interpretable generation dynamics without reliance on large embedding spaces or attention mechanisms.

4.7 Model Hyperparameters

GLP-Mono is defined by a compact set of hyperparameters that determine its geometry, paging structure, context representation, and sampling behaviour. All hyperparameters are fixed at initialization and remain constant throughout training and inference. Paging is fully static: the number of pages is fixed and does not grow dynamically. Likewise, the dimensionality of all input and output vectors is configurable and independent of the number of grammar-lemma pairs observed during training.

Table 1 summarizes the principal hyperparameters that define a GLP-Mono model.

Category	Hyperparameter
Paging	Number of pages (default: 256)
	Fixed paging; no dynamic page creation
Lemma Embeddings	Output embedding dimension (default: 256)
	Embedding update rate and damping schedule
Network Geometry	Hidden layer size of the monolithic MLP (default: 1536)
	Hidden layer size of the activation MLP (default: 64)
	Total input dimension (computed from context encoder)
	Total output dimension (pages + lemma embedding)
Context Encoding	History structure: 5 large sentence encodings, 15 medium sentence encodings
	Maximum lemmas for current sentence (default: 30)
	Narrative memory dimension (default: 128)
Sampling	Top- k sampling parameter (default: 20)
	Temperature (default: 0.8)

Table 1: Principal hyperparameters defining the GLP-Mono architecture.

These hyperparameters determine the complete geometry of GLP-Mono: the size of the monolithic network, the dimensionality of lemma embeddings, the structure of the context encoder, the number of pages, and the behaviour of the sampling mechanism. Because paging and output dimensions are fixed, vocabulary growth does not alter model size, and all grammar-lemma pairs discovered during training are mapped into the existing paged output space.

5 Inference

Inference in GLP-Mono is a deterministic, paged next-token process driven by a single feedforward network, a fixed-dimensional context encoder, and a learned set of grammar-lemma token pairs. Unlike transformer-based models, GLP-Mono does not rely on attention, softmax over a global vocabulary, or probabilistic routing. Instead, next-token prediction is executed through a sequence of structured operations: context encoding, monolithic network evaluation, page selection, cosine-based ranking, and top- k sampling.

5.1 Context Preparation

At each generation step, the model constructs a fixed-dimensional context vector consisting of:

1. history embeddings from previously generated sentences,
2. a medium-resolution embedding of the current partial sentence,
3. the recurrent narrative memory state,
4. the stable dataset sequence embedding associated with a keyword set.

This vector is supplied directly to the monolithic network. No attention mechanism or dynamic context expansion is used.

5.2 Network Evaluation

The monolithic MLP produces a single output vector that is split into:

1. a normalized lemma embedding $\hat{e}$,
2. a vector of page logits determining the relevance of each page.

The lemma embedding represents the model’s prediction of the next lexical content, while the page logits determine which cluster of grammar-lemma pairs is most appropriate given the current context.

5.3 Page Selection

Pages act as sparse MoE-style routing units. Inference proceeds by:

1. sorting pages by descending logit score,
2. selecting the highest-scoring page,
3. falling back to lower-ranked pages if the selected page contains no valid token pairs.

Terminal tokens (`<PERIOD>`, `<COMMA>`, `<EXCLAMATION>`, `<QUESTION>`, `<EOL>`) are implicitly available in every page and do not require routing.

5.4 Cosine-Based Ranking

Once a page is selected, GLP-Mono evaluates all grammar-lemma pairs (G, L) in that page by computing cosine similarity between the predicted lemma embedding $\hat{e}$ and the stored output embedding $e_{G,L}$:

$$s(G, L) = \cos(\hat{e}, e_{G,L}).$$

This ranking mechanism replaces the global softmax used in transformer models. Grammar tokens are deterministic symbols; lemma tokens are learned embeddings. No grammar filtering is applied: compatibility is encoded implicitly in the page structure learned during training.

5.5 Top- k Sampling

The ranked list of token pairs is truncated to the top- k candidates. Sampling proceeds by applying temperature-scaled softmax to their cosine scores:

$$p(G, L) = \frac{\exp(s(G, L)/T)}{\sum_{(G', L') \in \text{top-}k} \exp(s(G', L')/T)}.$$

A single grammar–lemma pair is drawn from this distribution. The grammar token is always emitted first; the lemma token is emitted only if it differs from the grammar token (terminals emit only the grammar symbol).

5.6 Sentence Finalization

When the grammar token is `<EOL>`, the model performs:

1. canonical grammar validation,
2. natural-language reconstruction,
3. semantic validation using the narrator’s quality filters.

If validation fails, the sentence is discarded and regenerated. If validation succeeds, the sentence is appended to the story and the context window is updated.

5.7 Deterministic Behaviour

Inference in GLP–Mono is fully deterministic except for the sampling step. Paging, grammar tokens, context geometry, and output dimensionality are fixed. Vocabulary growth does not alter model size, and all grammar–lemma pairs discovered during training are mapped into the existing paged output space.

This yields stable long-horizon behaviour, predictable narrative cadence, and domain-faithful generation even at small scale.

6 Training Regime

GLP–Mono is trained on a curriculum consisting of a finite set of stories. Each story is a sequence of canonical sentences, accompanied by a stable story embedding, and a keyword set derived from the nouns and verbs present in the entire story. These keywords serve two roles: they provide a compact semantic signature of the story domain, and they act as a steering mechanism during inference, allowing the narrator pipeline to select or bias world geometry without modifying model parameters.

6.1 Curriculum Structure

A curriculum story contains:

1. a sequence of canonical grammar–lemma sentences,
2. a deterministic story embedding computed from its token set,
3. a keyword set consisting of all nouns and verbs in the story.

The curriculum therefore defines the structural ontology of the domain: grammar cadence, lemma distribution, page formation, and narrative motifs.

6.2 Epoch Definition

Training proceeds in epochs, where *each epoch corresponds to exactly one curriculum story*. If the curriculum contains S stories, then one full pass over the curriculum consists of S epochs.

GLP-Mono uses two training phases:

- a **warmup phase** of W passes over the curriculum in story order,
- a **randomized phase** of M additional epochs.

The total number of epochs is therefore:

$$\text{epochs} = W \times S + M.$$

Warmup epochs traverse the curriculum in fixed order; randomized epochs select a single story uniformly at random.

6.3 Warmup Phase

During warmup, the model iterates over all stories in deterministic order. For each story:

1. The story embedding is injected into the context window.
2. Each sentence is processed sequentially:
 - the context vector is encoded,
 - the monolithic network predicts a lemma embedding and page logits,
 - a training sample is created containing the input vector, the grammar-lemma target pair, and the page index,
 - the context window is updated with the sentence embedding.

Warmup stabilizes grammar cadence, initializes page structure, and aligns predicted lemma embeddings with stored output embeddings.

6.4 Randomized Phase

After warmup, training continues for M epochs. At each epoch:

1. A random story is selected from the curriculum.
2. All sentences in the story are processed as in warmup.
3. Training samples are accumulated into a batch.

Randomization ensures that page structure, lemma embeddings, and narrative memory converge across all domains present in the dataset. This phase is responsible for *world blending*: multiple narrative domains become superimposed into a single coherent generative substrate.

6.5 Batch Learning

At fixed intervals, the accumulated batch is used to update the model.

Network Update. The monolithic MLP is trained using:

- cosine loss for lemma embeddings,
- cross-entropy loss for page selection for non-terminal targets,
- mean-squared error for page selection for terminal targets.

Lemma Embedding Update. After each batch, stored lemma embeddings are updated using a damped mixing rule:

$$e_{G,L} \leftarrow (1 - \alpha)e_{G,L} + \alpha\hat{e},$$

where $\hat{e}$ is the predicted lemma embedding and α decreases monotonically over training.

6.6 Page Formation

Pages are constructed dynamically during training. For each grammar–lemma pair:

1. The previous lemma token determines the routing key.
2. If a page exists for this key, the pair is added to that page.
3. If no page exists and the page limit is not reached, a new page is created.
4. If the page limit is reached, the pair is assigned to the smallest page.

This yields compact, domain-specific clusters of grammar–lemma pairs that serve as sparse routing units during inference.

6.7 Canonical Closure

After training, GLP–Mono forms a structural closure over the curriculum: the model compresses the grammar cadence, lemma distribution, page geometry, and narrative motifs of all stories into a single deterministic generative substrate. Distinct worlds become superimposed into a unified canonical geometry, enabling stable multi-world generation without drift.

The resulting model is a compact, domain-faithful canon engine that provides structural and semantic grounding for the narrator pipeline.

7 GLP–Mono as a Canon Engine and Narrative Substrate

After training, a GLP–Mono model becomes a compact and deterministic representation of the narrative domains present in its curriculum. The model does not memorize surface text, nor does it rely on guardrails, reinforcement learning, or externally imposed behavioural constraints. Instead, GLP–Mono forms a structural closure over the dataset: it compresses the grammar cadence, lemma distributions, page structure, and recurrent narrative patterns of all worlds it has observed into a single monolithic generative substrate.

Because paging organizes grammar–lemma token pairs into stable clusters, and because lemma embeddings converge through damped mixing, the trained model exhibits a form of *canon compression*. Distinct narrative domains present in the curriculum are superimposed into a single coherent generative geometry. Worlds do not interfere; instead, they coexist as overlapping attractors within the paged token space. During sampling, GLP–Mono navigates these attractors according to context, narrative memory, and page activation, producing text that reflects the structural ontology of the combined dataset rather than any individual source.

This behaviour makes GLP–Mono fundamentally different from transformer-based LMs. Transformers require large-scale embeddings, attention stacks, and alignment procedures to maintain coherence across domains. GLP–Mono requires none of these: its behaviour is entirely determined by the dataset and the deterministic geometry of its pages, embeddings, and grammar tokens. The model is not aligned, steered, or filtered; it is simply shaped by the worlds it has absorbed.

Within the narrator pipeline, GLP–Mono functions as a *world engine*. Its role is not to produce polished text, but to generate structurally coherent proto-stories that encode the canonical geometry of the learned domains. These proto-stories serve as creative groundstock: they provide the narrative

ontology, spatial motifs, character roles, and action grammar that define the world. A transformer-based LLM then performs interpretive refinement, smoothing transitions, forming arcs, and elevating style, while preserving the underlying world model supplied by GLP-Mono.

In this hybrid configuration, GLP-Mono effectively *steers* the LLM through context rather than through fine-tuning. The proto-stories act as a story bible: a structured, domain-faithful substrate that guides the LLM’s stylistic and narrative decisions without modifying its parameters. This enables world specialization without LoRA layers, without alignment procedures, and without architectural modification. GLP-Mono provides the world; the LLM provides the surface.

Thus, after training, GLP-Mono is best understood not merely as a language model, but as a deterministic canon engine: a compact, inspectable, and domain-stable substrate that reconstructs the worlds present in its curriculum and supplies them as creative context to downstream generative systems.

8 Conclusion

GLP-Mono introduces a structurally governed alternative to contemporary transformer-based language models. Its architecture is defined not by attention mechanisms or large embedding spaces, but by deterministic grammar tokens, paged grammar-lemma clusters, fixed-dimensional output geometry, and a single monolithic feedforward network. Through curriculum-driven training, GLP-Mono forms a canonical closure over all stories in the dataset: grammar cadence, lemma distributions, page structure, and narrative motifs are compressed into a stable generative substrate that reflects the structural ontology of the combined domains.

Because paging is fixed and vocabulary growth does not alter model geometry, GLP-Mono remains compact and predictable even as it absorbs multiple narrative worlds. Distinct domains become superimposed into a unified canonical space, yielding coherent multi-world behaviour without drift. Lemma embeddings converge through damped updates, pages stabilize into domain-specific clusters, and narrative memory provides long-horizon coherence without attention.

In the narrator pipeline, GLP-Mono functions as a world engine rather than a surface generator. It produces structurally coherent proto-stories that encode the learned world geometry, while a transformer-based model performs interpretive refinement. This hybrid configuration allows GLP-Mono to steer a large LLM through context rather than parameter modification, enabling world specialization without fine-tuning or alignment procedures.

The resulting system demonstrates that coherent, domain-faithful language generation does not require monolithic vocabularies or attention stacks. A single MLP, paired with deterministic grammar, paged lexical organization, and compact context encoding, is sufficient to produce stable, expressive, and interpretable next-token behaviour. GLP-Mono therefore represents a distinct architectural path: a modular, inspectable, and canon-stable foundation for structured generative systems.

A GLP-Mono and DMLG as a Distinct Architectural Lineage

GLP-Mono and DMLG do not belong to the broad class of “alternative transformer variants” that populate contemporary research. They represent a structurally divergent lineage: a family of language-centric systems built on deterministic geometry, paged lexical organization, and explicit governance rather than monolithic embeddings and attention. This appendix clarifies why these architectures are fundamentally distinct, and why they occupy a unique position within the landscape of language model design.

A.1 Beyond Transformer Variants

Most non-transformer proposals—state-space models, recurrent kernels, convolutional hybrids, mixture-of-experts routers—retain the core transformer assumptions: a single global vocabulary, a unified embedding space, probabilistic next-token prediction via softmax, and contextual structure encoded implicitly in hidden states. GLP-Mono and DMLG diverge from this paradigm at every structural level. They replace global embeddings with paged lexical clusters, softmax with cosine-ranking inside deterministic routing, and attention with grammar-driven compression and compact narrative memory.

A.2 Deterministic Grammar and Canon-Closed Geometry

Both architectures treat grammar as a symbolic, deterministic backbone rather than a learned statistical pattern. This yields predictable cadence, stable long-horizon structure, explicit constraints on lexical realization, and domain isolation through paging. Transformers do not provide deterministic grammar; GLP-Mono and DMLG do. This alone places them in a separate architectural category.

A.3 Paging as a Geometric Mechanism

Paging is not a routing trick or sparsity optimization. It is a geometric principle: a partitioning of lemma space into domain-specific clusters that remain stable across training. Paging prevents cross-domain drift, enables multi-world blending, and provides a structural substrate for canonical closure. No mainstream LM uses paging as a geometric constraint.

A.4 Canon Engines Rather Than Statistical Models

DMLG and GLP-Mono do not attempt to maximize fluency or mimic human text. They aim to reconstruct the structural ontology of a domain: grammar cadence, lemma distribution, narrative motifs, page geometry, and world attractors. Transformers memorize and interpolate; GLP-Mono and DMLG compress and reconstruct. This makes them canon engines rather than statistical predictors.

A.5 Hybridization Without Fine-Tuning

A transformer LLM can be steered by GLP-Mono purely through context, without modifying its parameters. This hybrid behaviour—world specialization without fine-tuning—is not available in transformer-only systems. It emerges from GLP-Mono’s deterministic geometry and canonical closure.

A.6 IEGR-Complete Structural Roles

Within the Triadic Cosmos framework, GLP-Mono and DMLG instantiate all four structural roles: energetic grammar decisions, geometric paging, informational lemma refinement, and recursive PRE-based evaluation. Transformers instantiate only the informational role at scale. GLP-Mono and DMLG therefore constitute a structurally complete substrate rather than a statistical engine.

A.7 Architectural Minimalism and Full Implementability

A striking property of both architectures is their implementational minimalism. The complete functional behaviour of GLP-Mono and DMLG—including grammar cadence, paging, context encoding, narrative memory, canonical closure, and next-token generation—can be expressed in a small number of Python classes. A typical reference implementation fits comfortably within a few thousand lines of code.

This stands in sharp contrast to transformer-based systems, whose operational behaviour depends on millions of lines of framework code, distributed training infrastructure, and opaque optimization pipelines. GLP-Mono and DMLG are architectures that can be fully understood, inspected, and modified by a single researcher. Their simplicity is not a limitation but a defining feature: they demonstrate that coherent world models, canonical geometry, and hybrid generative behaviour do not require large-scale infrastructure or monolithic parameter spaces.

A.8 Full Traceability and Non-Blackbox Behaviour

A defining characteristic of both GLP-Mono and DMLG is their complete traceability. Unlike transformer-based systems, whose internal behaviour emerges from billions of parameters distributed across opaque attention layers, GLP-Mono and DMLG expose every structural decision, routing step, and generative transition in a fully inspectable form. Their operational logic is not implicit in high-dimensional embeddings but explicit in deterministic grammar rules, paged lexical geometry, and small, modular components.

This traceability is not merely conceptual. The full functional behaviour of both architectures can be implemented in a compact set of Python classes. A typical reference implementation of GLP-Mono consists of:

- a grammar module defining deterministic structural tokens,
- a paging module managing lemma clusters and routing keys,
- a context encoder producing fixed-dimensional input vectors,
- a monolithic MLP with learned activation,
- a cosine-based ranking module for next-token selection,
- a narrative memory module with GRU-like updates,
- a sampling module implementing top- k selection,
- a training loop with damped lemma embedding updates.

DMLG is similarly compact: its canonical graph, paging biomes, local evaluators, grammar-driven compression, and PRE-based governance can be expressed in a small collection of classes that fit comfortably within a few hundred lines of code.

This minimalism stands in stark contrast to transformer-based systems, whose behaviour depends on extensive framework infrastructure, distributed training pipelines, and millions of lines of auxiliary code. GLP-Mono and DMLG are not blackbox engines: every transition, every routing decision, every page activation, and every grammar-lemma interaction is explicitly represented and fully inspectable. Their simplicity is a structural advantage, enabling researchers to understand, modify, and extend the architecture without relying on opaque emergent mechanisms.

The result is a class of language systems that are not only deterministic and modular, but also uniquely transparent. GLP-Mono and DMLG demonstrate that coherent world models, canonical geometry, and hybrid generative behaviour can be achieved without blackbox complexity, and that full interpretability is compatible with expressive next-token dynamics.

A.9 A Distinct Architectural Category

Taken together, these properties define a new class of language systems: deterministic grammar engines, paged lexical geometries, canonical world substrates, hybrid steering mechanisms, explicit governance layers, and multi-world attractor dynamics. GLP-Mono and DMLG are not transformer alternatives. They are structurally governed language architectures with their own geometric principles, training dynamics, and generative behaviour. They form a coherent lineage within the Triadic Cosmos ecosystem: a modular, inspectable, and canon-stable foundation for hybrid intelligent systems.

B GLP-Mono vs Transformer Language Models

This appendix provides a formal comparison between the *GLP-Mono* architecture and contemporary transformer-based language models.

B.1 Overview

Transformer LMs unify grammar, lexical content, and context into a single high-dimensional embedding space governed by self-attention. GLP-Mono, by contrast, performs next-token prediction through a single monolithic MLP whose output is split into:

1. a fixed-dimensional lemma embedding,
2. a one-hot page selector over a learned set of pages.

Each page contains a set of *grammar+lemma token pairs*. Terminals (<PERIOD>, <COMMA>, <EXCLAMATION>, <QUESTION>, <EOL>) are implicitly present in every page. Lemma selection is performed by cosine-similarity ranking between the predicted lemma embedding and the stored embeddings of all token pairs in the active page.

B.2 Structural Organization

- **Transformers:** A single global parameter space with no explicit separation between grammar and lexical content. Grammar is implicit in learned embeddings and attention patterns.
- **GLP-Mono:** Grammar tokens are deterministic symbolic tokens. Lemma tokens are represented as learned output embeddings. Next-token prediction produces a *pair* (G, L) directly. Grammar and lemma are not predicted by separate models; they are jointly represented in the output space.

B.3 Context Mechanisms

- **Transformers:** Context is maintained through multi-head self-attention over large hidden states.
- **GLP-Mono:** Context is encoded through grammar-based compression, sentence-level embeddings, and a compact recurrent narrative memory. The model receives a single fixed-dimensional context vector; no attention mechanism is used.

B.4 Vocabulary Geometry

- **Transformers:** Vocabulary size determines output dimension. Larger vocabularies require larger embedding matrices and softmax layers.
- **GLP-Mono:** Vocabulary growth does not alter model geometry. Lemma tokens are represented as fixed-dimensional learned embeddings. Grammar tokens remain symbolic. Pages partition the (G, L) pairs into compact clusters learned during training.

B.5 Routing and Modularity

- **Transformers:** Routing is implicit in attention weights; all tokens share the same embedding space and parameter matrices.
- **GLP-Mono:** Page logits provide sparse MoE-style routing. The highest-scoring page is selected; if no valid token pairs exist, the model falls back to the next page. Pages contain only (G, L) pairs observed during training. Terminals are always available.

B.6 Next-Token Prediction

- **Transformers:** Next-token prediction is performed over a monolithic softmax distribution across the entire vocabulary.
- **GLP-Mono:** Next-token prediction is performed as follows:
 1. The monolithic MLP outputs a lemma embedding and page logits.
 2. The highest-scoring page is selected (with fallback).
 3. All (G, L) token pairs in the page, plus the five terminal tokens, are scored by cosine similarity.
 4. Top- k sampling is applied to the ranked list.

No grammar filtering is applied. Grammar and lemma are not predicted separately; they are jointly encoded in the page structure.

B.7 Deterministic Geometry

- **Transformers:** Geometry scales with model size, vocabulary size, and attention width.
- **GLP-Mono:** Geometry is fixed and deterministic: grammar tokens are symbolic, lemma embeddings have fixed dimension, and pages provide domain isolation. Model size does not depend on vocabulary size.

B.8 Long-Horizon Behaviour

- **Transformers:** Long-horizon coherence depends on attention depth and training scale.
- **GLP-Mono:** Narrative memory provides compact recurrence; paging stabilizes domain-specific token clusters; grammar cadence enforces canonical structure. Long-horizon behaviour is stable even at small scale.

B.9 Summary

GLP-Mono and transformer LMs represent two fundamentally different approaches to language modeling. Transformers rely on large-scale statistical geometry and attention, while GLP-Mono relies on a monolithic MLP, paged token clusters, fixed-dimensional output embeddings, and deterministic grammar tokens. These differences make GLP-Mono a structurally governed, domain-isolated, and interpretable alternative to monolithic transformer designs.

C Comparison Between GLP-Mono and the DMLG Architecture

This appendix provides a unified comparison between GLP-Mono and the Dynamic Modular Language Graph (DMLG) architecture as developed across the structural papers in the Triadic Cosmos ecosystem. Although GLP-Mono inherits several structural ideas from DMLG, the two systems occupy fundamentally different architectural categories: DMLG is a deterministic, graph-based governance and world-absorption substrate, while GLP-Mono is a monolithic paged next-token language model.

C.1 Conceptual Foundations

DMLG. Across its architectural formulations, DMLG models language as traversal through a canonical graph of tokens and transitions. Tokens are canonical lemmas, grammar markers, or structural symbols; edges encode admissible transitions. Paging partitions the graph into modular biomes, enabling domain isolation, stylistic fidelity, and controlled traversal. Local evaluators score transitions, and sentence-level PRE (Propose–Retry–Evaluate) enforces structural validity. DMLG is not a language model: it is a structural governance substrate that can guide, constrain, or audit any generative system.

GLP–Mono. GLP–Mono is a monolithic next-token generator. It predicts a lemma embedding and page logits using a single feedforward network. Pages contain grammar–lemma token pairs discovered during training. Next-token selection is performed by choosing the highest-scoring page, ranking all token pairs by cosine similarity, and sampling from the top- k candidates. GLP–Mono is not a graph: it is a paged LM with fixed geometry and deterministic grammar.

C.2 Representation of Language

DMLG: Canonical Graph. Language is represented as an explicit graph $G = (V, E)$:

- V contains canonical tokens (lemmas, grammar markers, structural units),
- E contains admissible transitions,
- paging partitions V into modular biomes,
- traversal instantiates meaning through structured navigation.

This representation is deterministic, inspectable, and corpus-grounded.

GLP–Mono: Paged Token Pairs. Language is represented as grammar–lemma token pairs stored in fixed pages:

- grammar tokens are deterministic symbols,
- lemma tokens are learned output embeddings,
- pages cluster (G, L) pairs discovered during training,
- terminals are implicitly available in every page.

No graph structure or transitions exist; paging is used only for routing.

C.3 Generative Mechanism

DMLG: Graph Traversal. Generation is navigation through the canonical graph:

- local evaluators score transitions,
- grammar-driven context compression activates relevant regions,
- beam search explores candidate transition paths,
- PRE validates full sentences and triggers retries.

Generation is structural, not statistical.

GLP-Mono: Next-Token Prediction. Generation is next-token prediction:

- monolithic MLP outputs lemma embedding + page logits,
- highest-scoring page is selected (with fallback),
- cosine similarity ranks all token pairs in the page,
- top- k sampling selects the next token.

Generation is statistical, not structural.

C.4 Paging and Modularity

DMLG: Paging as Biomes. Paging defines modular subgraphs:

- pages represent semantic or structural biomes,
- traversal across pages is explicitly controlled,
- evaluators may be page-specific,
- paging provides domain isolation and stylistic fidelity.

GLP-Mono: Paging as Routing. Paging is fixed and used only for routing:

- pages cluster grammar-lemma pairs,
- no transitions exist between pages,
- paging does not encode structure, only selection,
- pages do not evolve after initialization.

C.5 Context and Memory

DMLG: Recursive Structural Context. DMLG maintains a rich structural context built from canonical grammar, recursive memory, and graph-based state tracking. Its context mechanisms include:

- grammar-driven context compression,
- recursive narrative memory and attractor-based world state,
- canonical graph state tracking across pages and transitions,
- evaluator-governed structural coherence during traversal.

This context is structurally expressive but distributed across multiple evaluators, paging biomes, and traversal states. As a result, context is explicit and modular, yet its influence on generation is mediated through many small components rather than a single learned function.

GLP-Mono: Learned Fixed-Dimensional Context Vector. GLP-Mono inherits the core context-encoding principles of DMLG—history compression, sentence embeddings, narrative memory, and keyword-derived story state—but integrates them into a single fixed-dimensional vector consumed by a monolithic network:

- history embeddings capturing long-range structural cadence,
- sentence embeddings encoding local grammar-lemma structure,
- a compact recurrent narrative memory tracking actors and events,
- a stable story embedding derived from keyword sets (nouns and verbs).

Because GLP-Mono is monolithic rather than modular, this context is not merely consulted but *learned*: the network internalizes how context influences next-token prediction. This yields a more effective and globally coherent use of context than in DMLG, where context is distributed across micro-models or graph evaluators. In GLP-Mono, the entire context vector participates directly in the learned mapping from narrative state to next-token behaviour, enabling stronger long-horizon coherence at small scale.

C.6 Functional Role in Hybrid Systems

DMLG: Governance and World Absorption. DMLG provides:

- canonical grammar and deterministic structural tokens,
- explicit transitions in a canonical graph,
- domain-locked behaviour through modular paging biomes,
- world absorption and stable attractors across long horizons,
- structural governance for LLMs through evaluator-based traversal and PRE.

Within the IEGR taxonomy, DMLG functions primarily as a **G-R substrate**: it supplies geometric structure (G) and recursive depth (R) through canonical grammar, graph traversal, paging biomes, and sentence-level PRE.

GLP-Mono: Creative Substrate with Sentence-Level PRE. GLP-Mono provides:

- structurally coherent proto-stories generated by a monolithic LM,
- canon-compressed world geometry learned from the curriculum,
- creative ground material for narrator pipelines and hybrid refinement,
- context steering for LLMs through stable embeddings and paged lexical structure,
- sentence-level PRE inherited from DMLG, ensuring structural validity.

Although GLP-Mono is monolithic and statistical rather than graph-based, it *retains sentence-level PRE*, making it not only an **I-G** substrate but also a lightweight **R substrate**. Its recursive behaviour emerges from the interaction between fixed-dimensional context encoding, recurrent narrative memory, and PRE-based sentence validation.

In hybrid systems, GLP-Mono therefore complements DMLG: DMLG supplies explicit structural governance and world absorption, while GLP-Mono supplies compact creative generation with learned context and retained recursive structure.

C.7 Summary

DMLG and GLP-Mono represent two structurally related but architecturally distinct approaches to controlled language generation. DMLG provides the structural foundations: canonical grammar, deterministic transitions, modular paging biomes, evaluator-governed traversal, and PRE-based sentence validation. It is a governance and world-absorption substrate designed for stability, domain fidelity, and explicit control.

GLP-Mono is the generative successor to DMLG. It inherits DMLG’s structural principles—deterministic grammar, paged lexical organization, context encoding, and sentence-level PRE—but integrates them into a single monolithic network that learns how context, paging, and lemma geometry interact. As a result, GLP-Mono provides:

- compact and efficient next-token generation,
- canon-compressed world geometry learned directly from the curriculum,
- structurally coherent proto-stories suitable for narrator pipelines,
- effective use of context through a learned fixed-dimensional vector,
- retained recursive behaviour through sentence-level PRE.

In this sense, GLP-Mono is not merely complementary to DMLG: it is a *descendant*. DMLG supplies explicit structure and governance; GLP-Mono distills these structural commitments into a compact, learned generative model that is far more effective for narrative production. Together, they form a coherent architectural lineage in which DMLG provides the structural substrate and GLP-Mono provides the generative engine.

D Integration in The Narrator Pipeline

The narrator pipeline is a hybrid generative system in which GLP-Mono provides the structural and creative substrate, while a transformer-based LLM supplies stylistic refinement, narrative shaping, and quality control. The pipeline operates in incremental stages, gradually constructing chapters from small GLP-generated units and refining them into polished literary output.

D.1 Incremental GLP Generation

Each chapter begins with a fresh context window and a set of steering keywords. GLP-Mono uses these keywords to select an appropriate internal embedding and then generates a sequence of short proto-sections. Each proto-section consists of multiple lines, and each line is chosen from several GLP-generated candidates. The selection criterion is intentionally simple—typically the longest or most coherent candidate—ensuring that GLP’s learned world model drives the creative content without interference from the LLM.

After each line is selected, it is canonically fixed and added to GLP’s context window. This creates a recursive memory loop: GLP continually builds upon its own output, maintaining world consistency and long-horizon coherence.

D.2 Keyword Evolution

At the end of each proto-section, the LLM proposes a new set of keywords based on the moderated content. These keywords are restricted to noun and verb lemmas and filtered by GLP-Mono to ensure domain fidelity. The updated keyword set guides the next proto-section, allowing the narrative to evolve while remaining anchored in GLP’s canonical world geometry.

D.3 Assembly of Proto-Material

A full chapter is constructed by generating several proto-sections in sequence. Each proto-section contributes a block of GLP-driven narrative, resulting in a larger proto-story composed entirely of GLP output. This proto-story expresses the world, characters, physics, and thematic attractors learned by GLP-Mono, but remains stylistically raw and structurally unpolished.

D.4 LLM Remastering

Once the proto-story is complete, the LLM performs a full remastering pass. This is not a minor grammatical correction: the LLM may reorganize sentences, adjust pacing, strengthen thematic cohesion, or reshape the narrative flow. The remastering is guided by a generic or domain-specific moderation prompt, ensuring that the final chapter adheres to the desired literary style while preserving the world and events defined by GLP.

The LLM then assigns a chapter title and evaluates the remastered chapter using a numerical score. If the score is insufficient, the remastered chapter is used as new input for another refinement cycle. This recursive moderation continues until a version is produced that meets the required quality threshold.

D.5 Summary

The narrator pipeline is therefore a two-layer generative system:

- **GLP-Mono** provides the structural creativity: world geometry, canonical grammar, narrative attractors, long-horizon coherence, and the full proto-story material from which chapters are built.
- The **LLM** provides stylistic refinement: keyword evolution, narrative remastering, title generation, and quality scoring.

All narrative content originates from GLP-Mono; the LLM shapes the expression without altering the underlying world. Because the GLP proto-story fully encodes the canonical geometry of the domain, the LLM does not require any world prompting to remain within the correct canon. Its remastering step rewrites the GLP material into fluent natural language while preserving the world, physics, characters, and motifs defined by GLP-Mono.

The result is a hybrid architecture capable of producing coherent, domain-faithful long-form narratives through structured recursion and controlled refinement, with GLP-Mono supplying the world and the LLM supplying the prose.

E Learned Activation Behaviour in GLP Models

GLP-Mono employs a learned activation function implemented as a compact three-layer MLP. Unlike fixed nonlinearities such as SiLU or GELU, the activation function in GLP models is not hand-designed: it is *emergent*. Its geometry is shaped entirely by the manifold of the training closure, the grammar cadence, the lemma distribution, and the attractor ecology of the domain.

E.1 Activation as Learned Nonlinear Geometry

The monolithic GLP network uses a learned activation σ_θ , parameterized by a small MLP with weights θ . During training, σ_θ evolves jointly with the main network:

$$f(x) = W_3 \sigma_\theta(W_2 \sigma_\theta(W_1 x)).$$

Because σ_θ is itself a neural network, its nonlinear geometry adapts to the structural constraints of the world closure. This produces activation functions that differ substantially from classical smooth nonlinearities: they may exhibit oscillatory behaviour, sharp curvature changes, damping regions, local

plateaus, or multi-modal transitions. These features arise naturally from the manifold of the domain and are not imposed externally.

E.2 Emergent Activation Dynamics

The learned activation function encodes several structural properties of the GLP world:

- **Semantic density:** high-density lemma regions induce sharper nonlinear transitions, reflecting frequent shifts in lexical geometry.
- **Grammar cadence:** deterministic grammar tokens constrain the activation shape, producing stable curvature patterns aligned with canonical sentence structure.
- **Attractor ecology:** stable motifs and recurrent structures generate smooth activation basins, while unstable attractors produce oscillatory or multi-modal behaviour.
- **Recursion depth:** narrative memory influences the activation geometry by shaping how past context affects current nonlinear routing.

These behaviours demonstrate that GLP activations are not trivial functions but *learned nonlinear laws* governing how energy flows through the network.

E.3 Activation as Local Physics

Within the GLP closure, the activation function acts as a form of *local physics*. Its geometry determines:

- how strongly next-token energy is amplified or damped,
- how paging geometry interacts with semantic transitions,
- how lemma embeddings evolve across recursion,
- how attractors stabilize or destabilize narrative flow.

In this sense, σ_θ is the nonlinear signature of the world: each GLP model learns its own activation physics, reflecting the manifold of its closure.

E.4 Triadic Interpretation

The learned activation function fits naturally into the Triadic Cosmos framework. Each activation update corresponds to a triadic cycle:

- **Energy (E):** next-token prediction produces activation input.
- **Geometry (G):** the learned activation defines nonlinear routing.
- **Information (I):** the output token embedding encodes semantic content.
- **Recursion (R):** narrative memory feeds back into the next cycle.

Because GLP–Mono is a local closure, the activation geometry becomes the world-specific instantiation of triadic physics: a learned nonlinear law governing how energy, geometry, information, and recursion interact inside the closure.

E.5 Summary

The activation MLP in GLP models is not a fixed nonlinearity but an emergent, domain-specific geometric structure. Its behaviour reflects the manifold of the world closure, encoding semantic density, grammar cadence, attractor ecology, and recursive dynamics. This learned activation physics is a defining feature of GLP-Mono and illustrates how compact LM architectures can internalize nontrivial nonlinear laws directly from the dataset manifold.

F Triadic Cosmos Ecosystem Context

The present paper is part of the broader *Triadic Cosmos* ecosystem: a unified research program that develops a compositional ontology of intelligence together with modular, inspectable computational architectures. The ecosystem spans conceptual theory, executable reference implementations, and fully documented empirical demonstrations, forming a coherent framework for studying hybrid intelligent systems.

All materials, extended notes, and evolving documentation are publicly available in the open GitHub project:

`https://github.com/triadic-cosmos`

Within the `triadic-system` repository, a complete reference implementation of the GLP engine, the narrator pipeline, and all supporting modules is provided under the AGPL-3.0 license. These components form the operational substrate for structured generation, world absorption, paging dynamics, and hybrid refinement.

A demonstration book, *The Monolithic Parrot: Multi-World Closures*, will also be part of the ecosystem. It presents a collection of unedited narrative outputs generated by the narrator pipeline across multiple domains, including single-world closures (e.g., science, Kafka) and multi-world blends derived from six distinct worlds. The book serves as a tangible illustration of GLP-Mono’s canonical geometry and its ability to steer a transformer LLM through context without world prompting, providing a practical showcase of hybrid generative behaviour within the Triadic Cosmos framework.

References

- [1] J. De Witte, “Grammar Lemma Paged Language Models,” Triadic Cosmos Ecosystem, 2026.
- [2] J. De Witte, “Learnable Lemma Embeddings for GLP Models,” Triadic Cosmos Ecosystem, 2026.
- [3] J. De Witte, “A Narrator Pipeline driven by DMLG,” Triadic Cosmos Ecosystem, 2026.
- [4] J. De Witte, “Language as Graph Traversal: The DMLG Architecture,” Triadic Cosmos Ecosystem, 2026.
- [5] J. De Witte, “A Dynamic and Modular Architecture for Language Modeling,” Triadic Cosmos Ecosystem, 2026.